

Paradigmi di Programmazione

Diego Stefanini — prof.ssa Chiara Bodei, a.a. 2026-27

Indice

1	Il lambda calcolo	3
1.1	Calcolabilità e paradigmi	3
1.2	Sintassi dei λ -termini	5
1.3	Variabili libere e legate	7
1.4	Sostituzione e β -riduzione	8
1.5	Confluenza e non terminazione	11

1 Il lambda calcolo

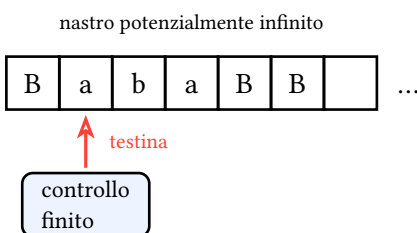
1.1 Calcolabilità e paradigmi

Hilbert, Entscheidungsproblem (problema della decisione): esiste una procedura **del tutto meccanica** che, data una qualunque formula della logica del primo ordine (un'affermazione matematica scritta in un linguaggio formale), dice se è un teorema, cioè se si può dimostrare?

Per rispondere bisogna prima dire cos'è una «procedura meccanica», cioè un **algoritmo**. Tre risposte diverse, quasi insieme:



Com'è fatta la **macchina di Turing**:

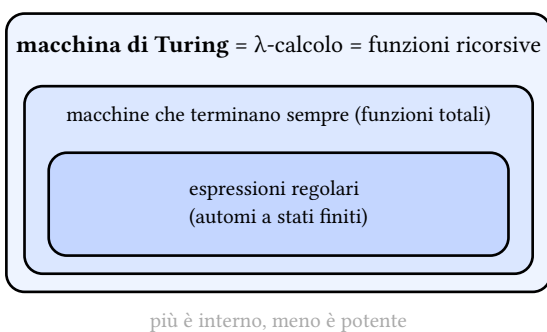


- Macchina **ideale**, non fisica: legge e scrive simboli su un nastro, secondo regole fissate.
- Esempio di regola: *legge a, riscrive a e si sposta a destra.*
- È il modello astratto di una macchina che esegue algoritmi.

Macchina di Turing Universale (UTM): una macchina di Turing che, data la **descrizione** di un'altra macchina e i suoi dati, la simula. È il modello teorico del computer: il programma è un dato.

Con la UTM Turing risponde **no** all'Entscheidungsproblem: esistono problemi che nessuna macchina di calcolo può risolvere.

I tre formalismi, anche se sono fatti in modo diversissimo, calcolano esattamente le stesse funzioni. Da qui la **tesi di Church-Turing**. Nella figura, i modelli meno potenti sono quelli che calcolano solo **funzioni totali** (danno sempre un risultato, per ogni input) e, ancora più dentro, le espressioni regolari:

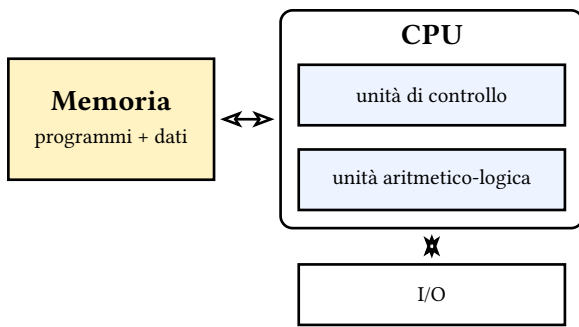


Se una funzione è calcolabile con un qualunque formalismo, allora esiste una macchina di Turing che la calcola.

È **indimostrabile** (i formalismi possibili sono infiniti) ma universalmente accettata: non si conosce nessun modello più potente.

Un linguaggio è **Turing completo** se calcola tutto ciò che calcola una macchina di Turing. Si dimostra scrivendo nel linguaggio un programma che **simula** una macchina di Turing.

Dai due modelli, Turing e Church, nascono due famiglie di linguaggi. Da Turing vengono i linguaggi **imperativi**, passando per la macchina di von Neumann:



Architettura di von Neumann, ispirata alla macchina di Turing ma concreta:

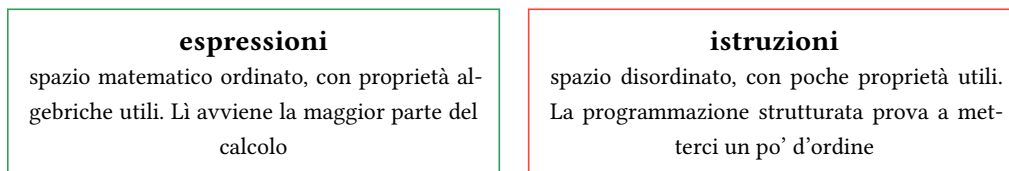
- **memoria** con programmi e dati;
- **CPU** che preleva un'istruzione alla volta, la interpreta e la esegue.

La generalità viene dal **programma memorizzato**: il programma sta in memoria come un dato qualsiasi, quindi la stessa macchina esegue qualunque programma basta caricarlo.

Un linguaggio «alla von Neumann» riproduce ad alto livello questa struttura:

Nel linguaggio		Nella macchina
variabili	↔	celle di memoria (il nastro)
istruzioni di controllo (if, cicli)	↔	test & jump (controllo una condizione e salto)
assegnamento	↔	modifica dello stato (leggo e scrivo la memoria)

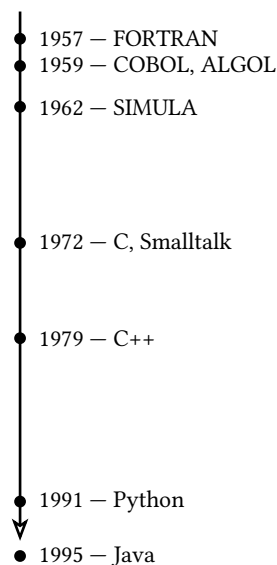
Backus: l'assegnamento divide la programmazione in due mondi.



Da Church vengono invece i linguaggi **funzionali**.

Programmazione funzionale: il programma è una serie di **valutazioni di funzioni matematiche**. Punto di forza: niente **effetti collaterali**. Una funzione ha un effetto collaterale quando, oltre a restituire un valore, cambia qualcosa fuori da sé (una variabile globale, un file, lo schermo). Senza effetti collaterali una funzione con lo stesso input dà sempre lo stesso output, quindi è più facile verificare che il programma sia corretto e ottimizzarlo. Il λ -calcolo (Church, 1935) è il primo linguaggio funzionale.

linguaggi «di Turing»



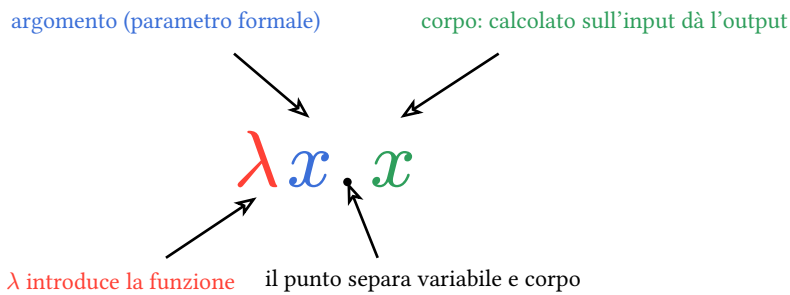
linguaggi «di Church»



Le due linee di discendenza

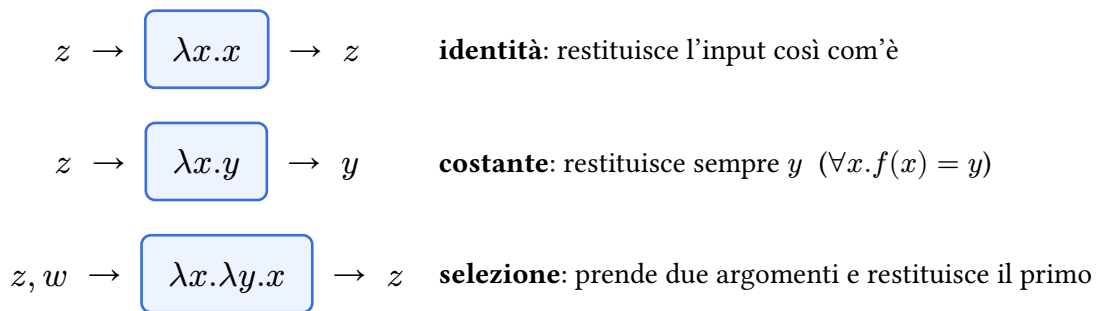
1.2 Sintassi dei λ -termini

Il λ -calcolo ha solo due operazioni. La prima è l'**astrazione**, cioè definire una funzione:



La **legge di corrispondenza** dice cosa fa la funzione su ogni input: qui $\forall x.f(x) = x$, cioè a ogni x associa x stesso. È la **funzione identità**.

La seconda è l'**applicazione**. Applicare una funzione = darle un **parametro attuale** al posto del parametro formale. Immagina la funzione come una scatola:



La selezione passo per passo: $((\lambda x.\lambda y.x)z)w \rightarrow (\lambda y.z)w \rightarrow z$. Nel primo passo metto z al posto di x nel corpo $\lambda y.x$; nel secondo metto w al posto di y nel corpo z , ma y non c'è, quindi w sparisce e resta z .

$\lambda x.\lambda y.x$ ha legge di corrispondenza $\forall x.f(x) = g$ dove $g(y) = x$: una funzione che restituisce una funzione. È un modo alternativo di scrivere $f(x, y) = x$ con una funzione di un solo argomento.

Queste due operazioni, più le variabili, sono tutto il linguaggio. Un programma è un'espressione (**λ -espressione**). Ci sono solo tre modi di costruirla:

$e ::= x$	variabile
$\lambda x.e$	astrazione funzionale (dichiarazione di funzione)
$e e$	applicazione (chiamata di funzione)

Niente altro! La sintassi è finita.

Si legge: un'espressione e è una variabile, **oppure** (λ) un'astrazione, **oppure** un'applicazione. La definizione è **ricorsiva**: dentro $\lambda x.e$ e dentro $e e$ ci sono altre espressioni, costruite con le stesse tre regole. Per esempio $(\lambda x.x)y$ è un'applicazione di $\lambda x.x$ (astrazione, con corpo la variabile x) alla variabile y .

$\lambda x.e$ è una **funzione anonima**: non ha nome, e x è la dichiarazione del suo parametro. Lo stesso concetto nei linguaggi:

Linguaggio	$x \mapsto x + 1$
JavaScript	<code>function(a){ return a + 1; }</code> oppure <code>a => a+1</code>
OCaml	<code>fun x -> x+1</code>
Java (si chiamano lambda)	<code>(int x) -> x + 1</code>

Da qui in poi useremo lettere come e, e_1, e_2, e_3 . **Non** sono variabili del λ -calcolo: sono nomi che stanno per **un'espressione qualsiasi**, come in algebra $a + b$ vale per due numeri qualsiasi. Così una regola scritta con e_1 ed e_2 vale per tutte le espressioni.

Per esempio $e_1 e_2$ vuol dire «un'espressione seguita da un'altra», cioè un'**applicazione**: e_1 è la funzione che chiamo, e_2 è l'argomento che le passo (il parametro attuale). In JavaScript si scriverebbe `e1(e2)`. La funzione non deve per forza avere un nome: la sua definizione può stare **direttamente dentro** la chiamata.

$$\underbrace{(\lambda x. (\lambda y. xy))}_{e_1 = \text{la funzione}} \quad \underbrace{(\lambda z. z)}_{e_2 = \text{l'argomento}}$$

Se si scrive tutto senza parentesi, però, non si capisce cosa va con cosa. $\lambda x. xy$ è la funzione $\lambda x. (xy)$ (prende x e restituisce x applicata a y) oppure $(\lambda x. x)y$ (l'identità applicata a y)? Per decidere, e per non scrivere troppe parentesi, valgono due convenzioni (importanti):

1. Lo scope di λ va il più a destra possibile

Il corpo di una λ è **tutto** quello che c'è dopo il punto, fino alla fine (o fino a una parentesi chiusa).

$\lambda x. xy$ è $\lambda x. (xy)$

$\lambda x. \lambda y. xy$ è $\lambda x. (\lambda y. (xy))$

2. L'applicazione associa a sinistra

Con più applicazioni di fila si parte da sinistra: prima e_1 applicata a e_2 , poi il risultato applicato a e_3 . Come `f(a)(b)` in JavaScript.

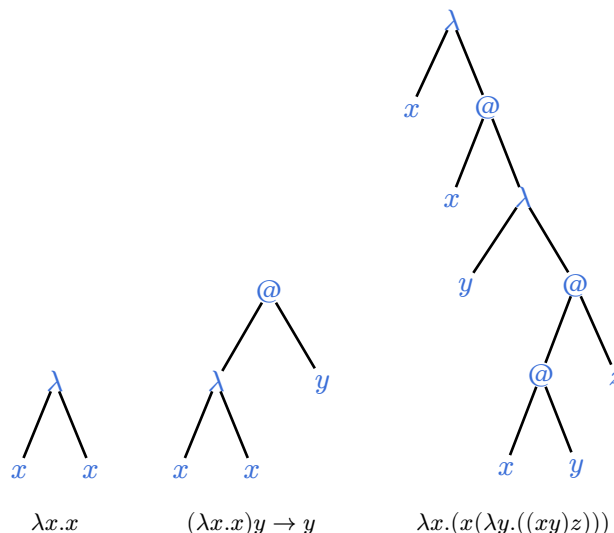
$e_1 e_2 e_3$ è $(e_1 e_2) e_3$

La selezione vista sopra ne è un esempio: $(\lambda x. \lambda y. x)zw$ è $((\lambda x. \lambda y. x)z)w$, cioè prima passo z , poi w .

Esercizio: quali parentesi sono sottintese in $\lambda x. x \lambda y. xyz$?

- Regola 1. Il corpo di λx è tutto il resto, $x \lambda y. xyz$; dentro, il corpo di λy è xyz : $\lambda x. (x \lambda y. (xyz))$
- Regola 2. xyz diventa $(xy)z$; e $x \lambda y. \dots$ è x applicata alla funzione $\lambda y. \dots$: $\lambda x. (x (\lambda y. ((xy)z)))$

Ogni termine si può disegnare come un **albero**, che segue l'annidamento completo delle parentesi. Serve a vedere quali passi di valutazione si possono fare. Nodo λ : figlio sinistro il parametro, destro il corpo. Nodo $@$ (applicazione): figli funzione e argomento.



1.3 Variabili libere e legate

λ è un **operatore di binding**: in $\lambda x.e$ lega la x dentro e (il suo **scope**), come $\forall x$ in $\forall x.P$ nella logica. È l'**unico** modo di associare valori a variabili nel λ -calcolo.

- **Legata**: introdotta da un λ che la contiene. **verde**, la freccia punta al suo λ .
- **Libera**: nessun λ la dichiara. **rossa**.

Espressione	Chi è legato
$\lambda \mathbf{x} . \mathbf{x}$	x legata
$\lambda \mathbf{x} . \lambda \mathbf{y} . (\mathbf{x} \mathbf{y} \mathbf{z})$	x e y legate, z libera
$(\lambda \mathbf{f} . \mathbf{f} \mathbf{x}) \mathbf{y}$	f legata, x e y libere
$(\lambda \mathbf{f} . \mathbf{f} \mathbf{x}) \mathbf{f}$	la prima f è legata; x e la seconda f sono libere : sta fuori dalle parentesi, fuori dallo scope di λf

Una stessa variabile può voler dire cose diverse in punti diversi:

$\lambda \mathbf{x} . \mathbf{x} (\lambda \mathbf{x} . \mathbf{x}) \mathbf{x}$

che con le parentesi esplicite è $\lambda x.(x(\lambda x.x)x)$

La x nel λx interno è legata a **quel** λ , non a quello esterno. Tre x , due significati: da qui nasce il conflitto di nomi.

Ecco il problema che questo crea. Applicando senza fare attenzione ai nomi:

	Espressione	Risultato
ingenuo	$(\lambda x.\lambda y.xy)y$	$\lambda y.yy$ ✗ sbagliato
rinomino y in z	$(\lambda x.\lambda z.xz)y$	$\lambda z.yz$ ✓

Le due espressioni di partenza sono **la stessa funzione** (ho solo cambiato il nome del parametro), ma i risultati si comportano in modo diverso. Nel primo caso la y libera che passo come argomento finisce **catturata** dal λy interno: è un **conflitto di nomi**.

x in $\lambda x.e$ è un segnaposto: si può rinominare con una variabile **fresca**, cioè un nome che non compare da nessuna parte nelle espressioni che stiamo trattando. È così che si risolve il conflitto di nomi, come nella riga «rinomino» della tabella.

Le variabili libere si possono definire in modo preciso, con una **definizione induttiva**: un caso per ogni elemento della sintassi.

$FV(x) = \{x\}$	variabile
$FV(e_1 e_2) = FV(e_1) \cup FV(e_2)$	applicazione: le libere di tutte e due
$FV(\lambda x.e) = FV(e) \setminus \{x\}$	astrazione: il λ lega x , la tolgo

FV sta per *Free Variables*. Una variabile che non è libera è **legata** (*bound*). Un termine senza variabili libere ($FV(e) = \emptyset$) si dice **chiuso**; i termini chiusi si chiamano **combinatori**, il più semplice è l'identità $\lambda x.x$.

Esercizio: $FV((\lambda x. \lambda y. xy)((\lambda z. z)k))$

$$= FV(\lambda x. \lambda y. xy) \cup FV((\lambda z. z)k) = \emptyset \cup \{k\} = \{k\}$$

Visto che un parametro è solo un segnaposto, cambiargli nome non cambia niente. $\lambda a. ac$ e $\lambda b. bc$ si dicono **α -equivalenti**: a e b non hanno un significato proprio, conta solo il **ruolo** che hanno nell'espressione. Lo stesso vale nei linguaggi:

`function(a){ return a + 1; }` è α -equivalente a `function(b){ return b + 1; }`

Espressioni α -equivalenti rappresentano **lo stesso programma**. Rinominare una variabile legata con una variabile fresca (che non compare nell'espressione) si chiama **α -conversione**. Serve a passare da $\lambda x. x$ a $\lambda z. z$, e soprattutto a togliere il conflitto di nomi visto sopra (*variable shadowing*):

$$\lambda \mathbf{x} . \mathbf{x} (\lambda \mathbf{x} . \mathbf{x}) \mathbf{x} \xrightarrow{\alpha} \lambda \mathbf{x} . \mathbf{x} (\lambda \mathbf{z} . \mathbf{z}) \mathbf{x}$$

i due termini sono equivalenti a meno di α -conversione, ma nel secondo ogni nome ha un solo significato

1.4 Sostituzione e β -riduzione

Cosa vuol dire **eseguire** (valutare) una λ -espressione? La valutazione, *eval*, fa solo una cosa: **chiamare funzioni**.

$\text{eval}((\lambda x. e_1)e_2)$: rimpiazzo ogni occorrenza di x in e_1 con e_2 , poi valuto il termine che ne risulta.

È la chiamata di una funzione: eseguo il corpo e_1 dopo aver messo il **parametro attuale** e_2 al posto del **parametro formale** x .

La **sostituzione** si scrive $e_1\{x := e_2\}$ (oppure $e_1\{e_2/x\}$): è e_1 con ogni occorrenza **libera** di x sostituita da e_2 . Per esempio $xz\{x := \lambda y. y\} = (\lambda y. y)z$.

Il problema: e se e_2 contiene una variabile libera che in e_1 è legata? Sostituendo alla cieca finisce **catturata** dal λ di e_1 , e legare una variabile libera cambia il significato. Esempio con le operazioni aritmetiche, per semplicità: $(\lambda x. (x * y))\{y := (x + x)\}$.

Come	Risultato
alla cieca	$\lambda x. (x * (x + x))$ ✗ le x di $x + x$ ora sono il parametro
α -conversione, z fresca	$(\lambda z. (z * y))\{y := (x + x)\} = \lambda z. (z * (x + x))$ ✓

La soluzione è la **sostituzione che evita la cattura** (*capture-avoiding substitution*): prima di sostituire, se serve, rinomino con l' α -conversione. La definizione ha un caso per ogni elemento della sintassi:

	Regola	Perché
variabile	$x\{x := e\} \equiv e$	è proprio la variabile da sostituire
	$y\{x := e\} \equiv y$ se $x \neq y$	un'altra variabile non si tocca
applicazione	$(e_1 e_2)\{x := e\} \equiv (e_1\{x := e\})(e_2\{x := e\})$	sostituisco da tutte e due le parti
astrazione	$(\lambda x. e_1)\{x := e\} \equiv \lambda x. e_1$	qui x è legata: la sostituzione vale solo per le variabili libere, quindi non cambia niente
	$(\lambda y. e_1)\{x := e\} \equiv \lambda y. (e_1\{x := e\})$ se $y \neq x$ e $y \notin FV(e)$	in e non compare y : nessun rischio di conflitto
	$(\lambda y. e_1)\{x := e\} \equiv \lambda z. ((e_1\{y := z\})\{x := e\})$ se $y \neq x$ e $y \in FV(e)$, z fresca	la y libera di e verrebbe catturata: prima α -converti y in z , poi sostituisco

«Fresca» qui vuol dire che non compare né in e_1 né in e . L'ultimo caso, disegnato: sostituire x con y nel corpo di $\lambda x. \lambda y. xy$.

$\lambda \textcolor{green}{y} . \textcolor{green}{y} \textcolor{green}{y}$

alla cieca: la y arriva in un mondo dove esiste
già una y legata e diventa indistinguibile da lei $\times$

$\lambda \textcolor{green}{z} . \textcolor{red}{y} \textcolor{green}{z}$

prima rinomino y in z :
ora la y resta libera $\checkmark$

Un esempio completo: $(\lambda x. \lambda y. ((\lambda z. z)x))y$. Il passo $(\lambda y. ((\lambda z. z)x))\{x := y\}$ fatto alla cieca **non va bene**: le due y sono diverse ($y \in FV(e)$). Quindi, con k fresca:

$$(\lambda y. ((\lambda z. z)x))\{x := y\} = (\lambda k. ((\lambda z. z)x))\{x := y\} = \lambda k. ((\lambda z. z)y)$$

Esercizi. L'espressione è sempre la stessa, cambia la variabile da sostituire ($\equiv_\alpha$ vuol dire « α -equivalente», cioè uguale a meno di rinominare):

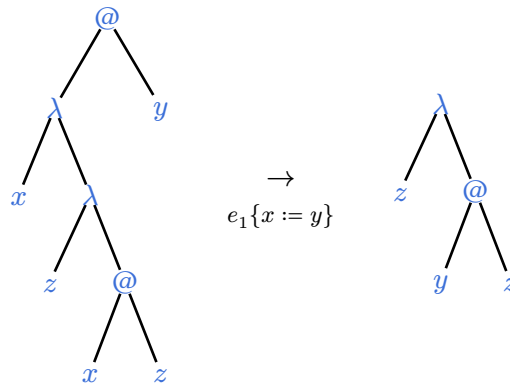
Sostituzione	Risultato
$((\lambda x. yx)w)\{x := \lambda k. kx\}$	$(\lambda x. yx)w$ l'unica x è legata: $(\lambda x. e_1)\{x := e\} \equiv \lambda x. e_1$
$((\lambda x. yx)w)\{y := \lambda k. kx\}$	$\equiv_\alpha ((\lambda z. yz)w)\{y := \lambda k. kx\} = (\lambda z. (\lambda k. kx)z)w$ la x di $\lambda k. kx$ è libera e verrebbe catturata da λx : rinomino
$((\lambda x. yx)w)\{w := \lambda k. kx\}$	$(\lambda x. yx)(\lambda k. kx)$ w non è sotto nessun λ : caso applicazione

Con la sostituzione si scrive la regola fondamentale del λ -calcolo, quella che lo rende un modello di calcolo universale, la **β -riduzione**:

$$(\lambda x. e_1)e_2 \rightarrow e_1\{x := e_2\}$$

- Cattura esattamente l'**applicazione di funzione**: $(\lambda x. x)3 \rightarrow 3$.
- Il risultato è il corpo e_1 in cui il parametro formale x è sostituito da copie dell'argomento e_2 .
- Un **redex** (espressione riducibile) è una sottoespressione della forma $(\lambda x. e_1)e_2$, a cui la regola si può applicare.

Esempio: $(\lambda x. \lambda z. xz)y$. Il parametro formale è x , il corpo è $e_1 = \lambda z. xz$, il parametro attuale è $e_2 = y$:



$$(\lambda x. \lambda z. xz)y \rightarrow \lambda z. yz$$

Il λ -calcolo è di **ordine superiore**: una funzione può prendere funzioni come parametri e restituire funzioni come risultato, in modo naturale.

Espressione	Cosa fa
$\lambda x. x$	identità
$\lambda y. (\lambda x. x)$	scarta l'argomento y e restituisce l'identità: è una funzione costante
$\lambda f. f(\lambda x. x)$	data una funzione f , la invoca sull'identità

Applicazione	Passi
$(\lambda x. x)y$	$\rightarrow y$
$(\lambda x. x)(\lambda y. y)$	$\rightarrow \lambda y. y$ l'identità applicata all'identità
$(\lambda x. xy)z$	$\rightarrow zy$ prende una funzione z e la applica a y
$(\lambda x. xy)(\lambda z. z)$	$\rightarrow (\lambda z. z)y \rightarrow y$ il parametro attuale è l'identità
$(\lambda x. ((\lambda y. y)x))z$	$\rightarrow (\lambda y. y)z \rightarrow z$
$(\lambda f. fz)(\lambda x. x)$	$\rightarrow (\lambda x. x)z \rightarrow z$ prende una funzione e la applica a z
$(\lambda x. (xx))(\lambda y. y)$	$\rightarrow (\lambda y. y)(\lambda y. y) \rightarrow \lambda y. y$ due termini uguali con ruoli diversi: funzione e argomento
$((\lambda x. \lambda y. x + y)3)5$	$\rightarrow (\lambda y. 3 + y)5 \rightarrow 3 + 5 \rightarrow 8$ (supponendo di avere il +)
$((\lambda x. \lambda y. xy)(\lambda x. x))z$	$\rightarrow (\lambda y. (\lambda x. x)y)z \rightarrow (\lambda x. x)z \rightarrow z$ forma $e_1 e_2 e_3$
$(\lambda x. \lambda y. xy)zk$	$\rightarrow (\lambda y. zy)k \rightarrow zk$

$(\lambda y. 3 + y)$ è una funzione a sé: somma 3 a quello che le passi. Applicando una funzione di due argomenti a uno solo ottieni una funzione che aspetta l'altro.

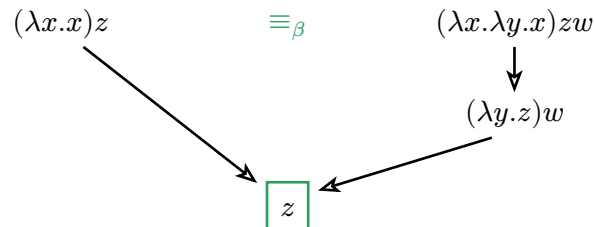
La valutazione va avanti scegliendo un redex e riducendolo. Quando non ci sono più redex l'espressione è in **forma normale β** : non si può più riscrivere con la β -riduzione, ed è il **risultato finale**, il **valore calcolato**. Per esempio $\lambda x. x$ e $\lambda t. \lambda f. t$ sono valori: **le funzioni sono valori**.

Notazione	Significato
$e_1 \rightarrow e_2$	e_2 si ottiene da e_1 con un passo di riduzione
$e_1 \Rightarrow e_2$	e_2 si ottiene da e_1 con zero o più passi

Un passo di β -riduzione è un passo di calcolo, quindi $\Rightarrow$ (la **chiusura riflessiva e transitiva** di $\rightarrow$) rappresenta una computazione qualsiasi. Quando $e_1 \Rightarrow e_2$ si dice che e_1 è **β -riducibile** a e_2 .

Con $\Rightarrow$ si definisce quando due espressioni sono «uguali». e_1 ed e_2 sono **β -equivalenti**, $e_1 \equiv_\beta e_2$, se:

1. sono identiche a meno di α -conversione, oppure
2. $e_1 \Rightarrow e_2$ oppure $e_2 \Rightarrow e_1$, oppure
3. $e_1 \Rightarrow e$ e anche $e_2 \Rightarrow e$ (arrivano alla stessa espressione).

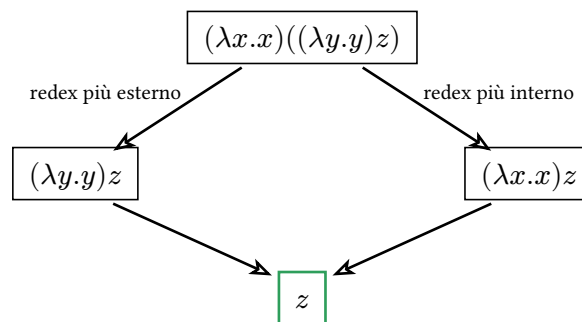


β -equivalenti per il terzo caso: tutte e due si riducono a z

Intuizione: due espressioni sono β -equivalenti quando sono indistinguibili dal punto di vista del calcolo, cioè calcolano gli stessi risultati.

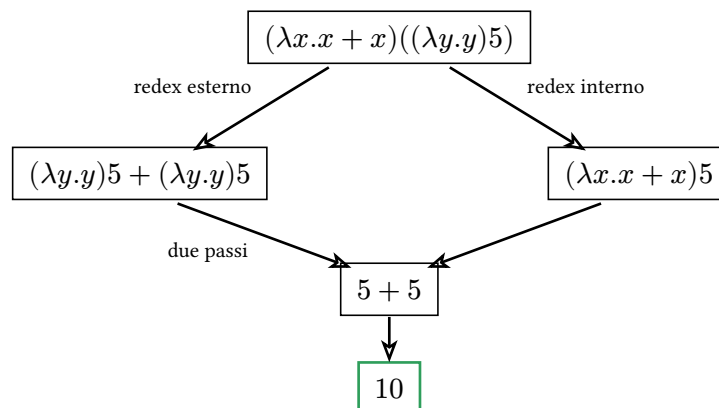
1.5 Confluenza e non terminazione

Quando un'espressione contiene più redex, si può scegliere da quale cominciare. In $(\lambda x.x)((\lambda y.y)z)$ ce ne sono due:



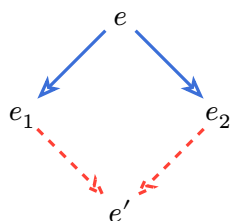
Strade diverse, stesso risultato

Ma in generale l'ordine di valutazione può cambiare il risultato finale? Un altro esempio (usando il +):



Scegliendo prima il redex esterno si copia $(\lambda y.y)5$ e lo si riduce due volte; prima quello interno, una volta sola. Il risultato è lo stesso

La risposta generale è il **teorema di Church-Rosser**: l'ordine in cui si scelgono le β -riduzioni **non influisce sul risultato finale**. Più precisamente: se a una stessa espressione si possono applicare due riduzioni (o sequenze di riduzioni) diverse, esiste un'espressione raggiungibile da entrambi i risultati con altre riduzioni (anche nessuna).



Come due strade diverse fra le vie di una città che partono dallo stesso punto e : qualunque giro si faccia, ci si può sempre ritrovare nello stesso punto e' .

Per questo nell'esempio sopra entrambe le strade arrivano a 10.

Proprietà di confluenza
(o del diamante)

Non sempre però si arriva a una forma normale. Il combinatore

$$\Omega = (\lambda x.xx)(\lambda x.xx)$$

contiene un solo redex, e ridurlo dà di nuovo Ω : la funzione $\lambda x.xx$ applica il suo argomento a se stesso, e l'argomento è proprio $\lambda x.xx$.

$$(\lambda x.xx)(\lambda x.xx) \xrightarrow{\beta} (\lambda x.xx)(\lambda x.xx) \longrightarrow \dots$$

loop! è di nuovo Ω

Ω non si può ridurre in forma normale: è un combinatore **divergente**, l'equivalente di un ciclo infinito.

Una stessa espressione può **terminare** facendo certe scelte di riduzione e **non terminare** facendone altre. Per esempio $(\lambda x.y)\Omega$:

$$(\lambda x.y)\Omega \xrightarrow{y} (\lambda x.y)\Omega \xrightarrow{y} (\lambda x.y)\Omega \xrightarrow{y} (\lambda x.y)\Omega \longrightarrow \dots$$

In orizzontale riduco dentro Ω e resto sempre fermo; in qualunque momento posso invece applicare $\lambda x.y$, che butta via l'argomento e dà y

Church-Rosser garantisce che **in tutti i casi in cui la riduzione termina, il risultato è lo stesso**: non possono esserci risultati diversi.